Look DNA — PRD & Work Split

Nov 1, 2026 · @Utpal

Overview

Look DNA turns any fashion or beauty inspiration image into a complete, purchasable look adapted to the user's skin tone and budget, shown on the user before they buy. We submit to the eCommerce VTO track of the YouCam API Skin AI & eCommerce VTO Hackathon, deadline 2 Nov 2026, 10:15pm IST.

One-line pitch: Screenshot any look. We find the pieces, adapt the shades to your skin tone, fit it to your budget, and show it on you before you buy.

Problem: Shoppers save looks from Instagram and Pinterest but struggle to recreate them. Try-on tools and product recommendations are fragmented, the original look was usually worn by someone with a different skin tone, and the original pieces are often over budget.

Target user: Shoppers aged 18 to 30 who save outfit and makeup inspiration and buy online on a budget, especially those whose skin tone rarely appears in the looks they save.

Why it fits the judging: It combines beauty and fashion try-on in one shopping flow (the VTO track brief), uses three YouCam APIs meaningfully, runs an agentic pipeline, and carries an inclusion story like the round-one winners.

Goals, non-goals and success metrics

The goal is one complete flow, from inspiration image to try-on to cart, working on a live URL with real YouCam calls.

Goals

- A working, deployed web app that a first-time user understands in under 10 seconds.
- At least three YouCam APIs used: Skin Tone Analysis, Clothes Virtual Try-On, Makeup Virtual Try-On.
- Makeup shades adapted to the user's undertone, with a one-line reason per shade.
- A total look price that stays under the user's budget.

Non-goals (cut from the original idea)

- Transform as a standalone feature, Evolve personas, hair and jewelry try-on, user accounts, payments.
- Live product search across the web. We use a curated catalog and say so.

Success metrics (measured on a 10-image test, reported in the video and README)

Metric	Target
Look items matched to a catalog product	80% or more
Total look cost vs budget	Under budget in 10 of 10 runs
Time from upload to try-on result	Under 60 seconds
Poor-quality selfies caught by the quality gate	All test cases

User flow and functional requirements

The flow is six screens in a straight line, each with one job.

1. **Upload inspiration:** the user uploads or pastes a screenshot of a look they love.
2. **Upload selfie + budget:** the user takes or uploads a selfie and sets a budget (in INR by default).
   - The quality gate checks brightness, blur and that one face is visible. A failed check shows a specific retake message.
3. **Look DNA card:** shows the extracted elements (top, bottom or dress, lip, eye, cheek) with color swatches and style tags.
4. **Adapted to you:** shows the user's skin tone and undertone, and each makeup shade before and after adaptation, with a one-line reason.
5. **Try-on result:** the full look on the user's selfie, clothes and makeup together. A toggle shows the original inspiration side by side.
6. **Cart:** matched products with image, brand, price and buy link, the total against the budget, and a "similar, not identical" label with a match score on each item. A swap button replaces one item with the next best match.

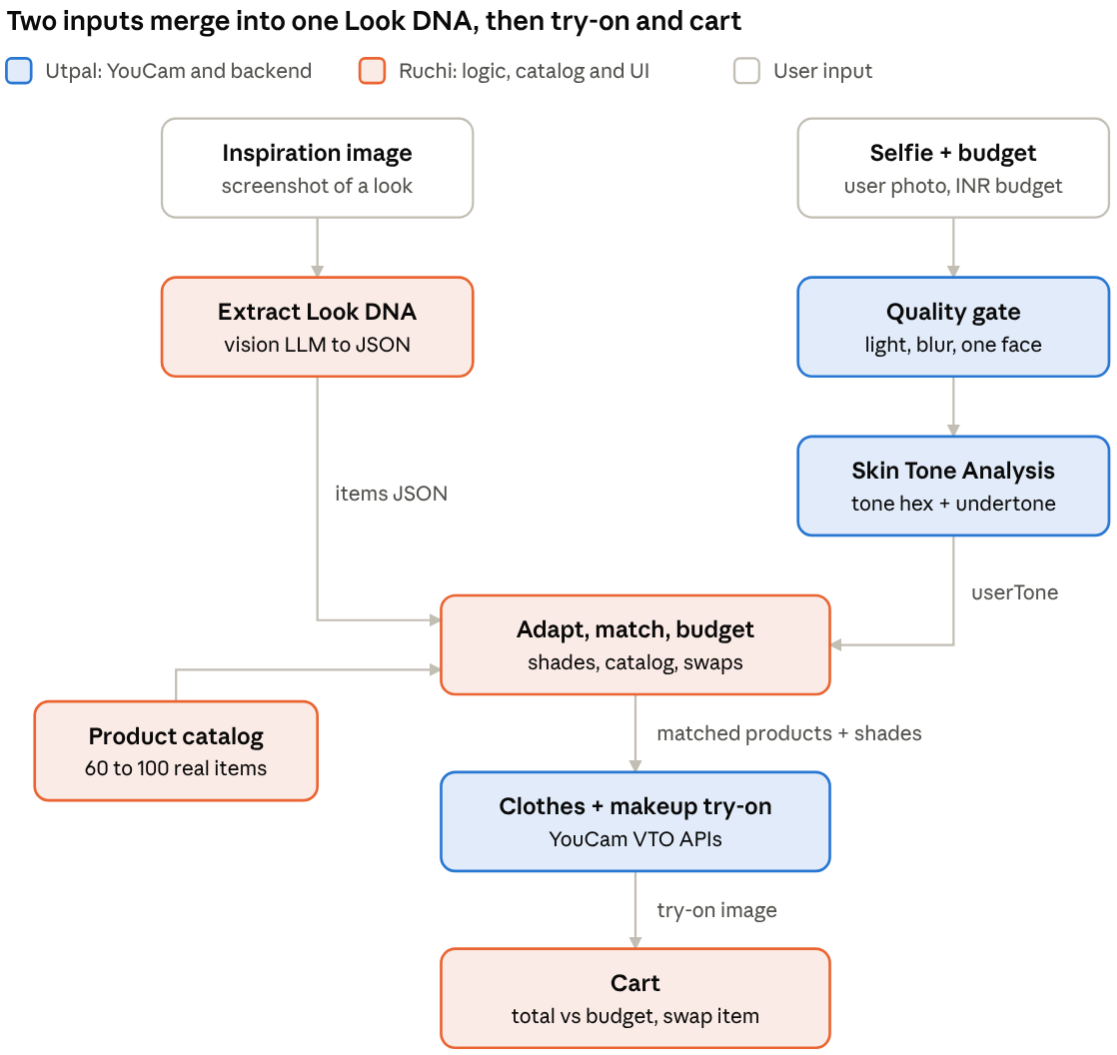
Requirements

ID	Requirement	Owner
F1	Extract look elements from the inspiration image as JSON	Ruchi
F2	Reject poor selfies before calling YouCam	Utpal
F3	Return skin tone hex and undertone from Skin Tone Analysis	Utpal
F4	Adapt makeup shades to undertone, with a reason	Ruchi

ID	Requirement	Owner
F5	Match each element to a catalog product by color and style	Ruchi
F6	Keep the total under budget by swapping the priciest item	Ruchi
F7	Render clothes try-on with the matched product image	Utpal
F8	Render makeup try-on with the matched shades	Utpal
F9	Cache every YouCam result by image hash	Utpal
F10	Six-screen UI, mobile-first, with loading states (Utpal builds screens 2 and 5)	Ruchi

Architecture, YouCam APIs and data contract

The app is one Next.js project on Vercel, and Ruchi's logic meets Utpal's YouCam routes only at the Look DNA JSON, so neither blocks the other.



Look DNA architecture · 9 parts, 2 owners

The selfie path is Utpal's, the inspiration path and catalog are Ruchi's, and they meet at the matching step.

YouCam APIs used (confirm exact endpoint names and the async task flow in the YouCam docs)

API	Used for	Owner
AI Skin Tone Analysis	Skin tone hex and undertone for shade adaptation	Utpal
AI Clothes Virtual Try-On	Matched top or dress on the selfie, from the catalog product image	Utpal
AI Makeup Virtual Try-On	Matched lip, eye and cheek shades on the selfie	Utpal

A vision LLM handles extraction (Ruchi). The YouCam key stays server-side only.

Data contract (commit as a shared types file in hour one)

```
{
  "items": [
    {"category": "top", "color": "#C8A2C8", "style": ["oversized", "knit"]},
    {"category": "lipstick", "color": "#B5485D", "finish": "matte"}
  ],
  "userTone": {"hex": "#C68E6B", "undertone": "warm"},
  "budget": 3000,
  "matches": [
    {"category": "lipstick", "productId": "lip-014", "adaptedColor":
"#A8473F",
     "reason": "A warmer red suits your warm undertone", "score": 0.86,
"price": 499}
  ],
  "total": 2850
}
```


Catalog item


```
{"id": "lip-014", "name": "", "brand": "", "category": "lipstick", "price":
499,
 "image": "https://...", "color": "#A8473F", "style": ["matte"], "link":
"https://..."}
```


Frontend and backend

One Next.js app holds both: the frontend is six React screens, and the backend is five API routes running serverless on Vercel. Utpal owns the YouCam routes and two screens; Ruchi owns the logic routes and four screens.

Stack

Layer	Choice	Owner
Frontend	Next.js App Router, React, Tailwind, mobile-first	Ruchi (Utpal: screens 2 and 5)
Backend	Next.js API routes, serverless on Vercel	Utpal and Ruchi
AI services	YouCam APIs; a vision LLM for extraction	Utpal; Ruchi
Data	`catalog.json` in the repo, no database, no accounts	Ruchi

Layer	Choice	Owner
Cache	Results keyed by image hash, in memory or Vercel KV	Utpal
State	Flow state in React context on the client	Ruchi

Backend: API routes

Route	Input to output	Calls	Owner
`POST /api/quality`	Selfie to pass or a retake message	Local brightness and blur check	Utpal
`POST /api/skin-tone`	Selfie to `userTone`	YouCam Skin Tone Analysis	Utpal
`POST /api/extract`	Inspiration image to `items`	Vision LLM	Ruchi
`POST /api/match`	`items`, `userTone`, budget to `matches`, `total`	`catalog.json` only	Ruchi
`POST /api/tryon`	Selfie and `matches` to try-on image	YouCam Clothes VTO, then Makeup VTO	Utpal

Try-on chains clothes first, then makeup on that output. If chaining fails, show the clothes and makeup results side by side.

Frontend: screens

Screen	Route	Main components	Calls	Owner
1 Upload inspiration	`/`	ImageDrop	none	Ruchi
2 Selfie and budget	`/selfie`	CameraCapture, BudgetInput, RetakeBanner	`/api/quality`	Utpal
3 Look DNA card	`/dna`	DnaCard, Swatch, StyleTag	`/api/extract`	Ruchi
4 Adapted to you	`/adapt`	ToneBadge, ShadeBeforeAfter	`/api/skin-tone`, `/api/match`	Ruchi

Screen	Route	Main components	Calls	Owner
5 Try-on result	`/tryon`	TryOnViewer, CompareToggle, LoadingSteps	`/api/tryon`	Utpal
6 Cart	`/cart`	CartItem, MatchScore, SwapButton, BudgetBar	`/api/match`	Ruchi

Folder structure

```
app/
  page.tsx                  screen 1 (Ruchi)
  selfie/  tryon/           screens 2, 5 (Utpal)
  dna/  adapt/  cart/       screens 3, 4, 6 (Ruchi)
  api/
    quality/route.ts        (Utpal)
    skin-tone/route.ts      (Utpal)
    tryon/route.ts          (Utpal)
    extract/route.ts        (Ruchi)
    match/route.ts          (Ruchi)
lib/
  types.ts                  shared data contract (both)
  youcam.ts                 upload, run task, poll (Utpal)
  cache.ts                  (Utpal)
  llm.ts                    (Ruchi)
  matching.ts               color distance, budget swaps (Ruchi)
data/
  catalog.json              (Ruchi)
  mocks/                    saved YouCam responses (Utpal)
```

Work split

Utpal owns everything that touches YouCam and the server; Ruchi owns the Look DNA logic, catalog, UI and story. You only depend on each other through the data contract, so work in parallel from hour one.

Together, first hour

- [ ] Create the repo and Next.js project, deploy an empty app to Vercel
- [ ] Agree on the data contract in the Architecture section and commit it as a shared types file
- [ ] Collect 5 inspiration images and 2 selfies (one well lit, one dark) for testing
- [ ] Confirm each of you has redeemed your 1,000 YouCam units

Utpal: YouCam integration, backend, deployment

- [ ] Server-side API routes with the YouCam key in environment variables only
- [ ] Quality gate: brightness, blur and face check, with retake messages (F2)
- [ ] Skin Tone Analysis route returning `userTone` in the contract format (F3)
- [ ] Clothes try-on route using the catalog product image as the garment (F7)
- [ ] Makeup try-on route for lips first, then eyes or cheeks (F8)
- [ ] Async task polling, timeouts and clear error states, following the YouCam docs
- [ ] Cache results by image hash so the same photo never spends units twice (F9)
- [ ] Mock mode: a flag that returns saved responses, so Ruchi can build without spending units
- [ ] Screens 2 and 5: selfie capture with retake messages, and the try-on viewer with compare toggle
- [ ] README setup steps and the architecture diagram

Ruchi: Look DNA logic, catalog, UI, story

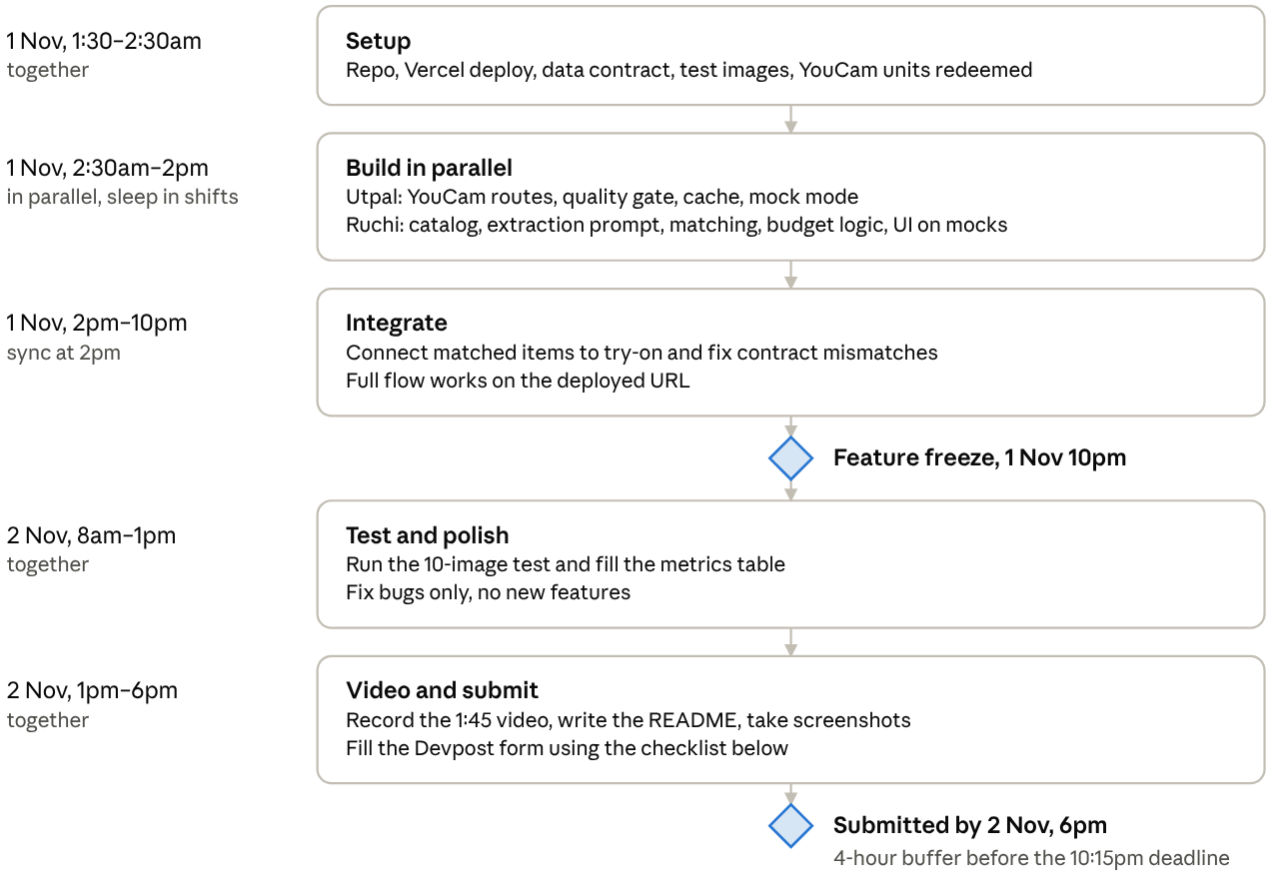
- [ ] Catalog JSON of 60 to 100 real products: name, brand, category, price, image, shade hex, buy link
- [ ] Extraction prompt: a vision LLM turns the inspiration image into `items` JSON (F1)
- [ ] Undertone adaptation rules with a one-line reason per shade (F4)
- [ ] Matching as plain code: color distance plus style-tag overlap, top 3 per item (F5)
- [ ] Budget logic: pick the best match per item, swap the priciest for the next cheaper match until under budget (F6)
- [ ] Screens 1, 3, 4 and 6 plus shared layout and styles, mobile-first, built against mock mode (F10)
- [ ] Devpost description, screenshots and the 1:45 video script

Together, final day

- [ ] Run the 10-image test and fill the metrics table
- [ ] Record the video, both of you on screen or voice
- [ ] Submit on Devpost

Timeline and submission checklist

All times are IST. Freeze features at 10pm on 1 Nov and submit by 6pm on 2 Nov, leaving a 4-hour buffer.



Build plan · 5 phases, 2 gates, IST

If the full flow is not working by the 10pm freeze, apply the fallbacks in Risks before adding anything.

Submission checklist

- [ ] Repo public with a license, or private and shared with contact_event@PerfectCorp.com
- [ ] README: setup steps, YouCam APIs used, architecture, 10-image test results
- [ ] Devpost text description covering features and consumer and retail value
- [ ] Screenshots of all six screens
- [ ] 1 to 3 minute video, public on YouTube, naming the YouCam APIs and showing the app running on the device
- [ ] No copyrighted music or third-party trademarks in the video
- [ ] Women in Tech award eligibility reflected in the registration, if it applies
- [ ] Demo video shared online for bonus API units

Risks and fallbacks

The biggest risk is running out of time with a half-working flow, so every risk below has a cut that keeps the flow complete.

Risk	Fallback	Owner
Makeup try-on integration takes too long	Ship clothes try-on plus lipstick only	Utpal
Running out of API units	Mock mode during development, cache everything, use the Share & Earn bonus	Utpal
YouCam calls are slow or time out in the demo	Pre-run the demo images and serve cached results in the video	Utpal
LLM extraction returns messy JSON	Strict JSON-only prompt, schema validation, one retry	Ruchi
Catalog matches look poor	Narrow to 3 categories (top, dress, lipstick) with denser coverage	Ruchi
Clothes try-on fails on some product photos	Use only catalog items with clean, front-facing product images	Ruchi
Video runs over 3 minutes	Script to 1:45, cut setup and talk only over working footage	Ruchi

Rule: no new features after the 10pm freeze on 1 Nov.